

Lecture 6

Integrating quality
activities in the project
life cycle

Outline

- Factors affecting intensity of quality assurance activities
- Testing level
- A model for SQA defect removal effectiveness and cost

References

- Daniel Galin, Software Quality Assurance: From theory to implementation, chapter 7.2-7.4, 10
- Glenford J. Myers, The Art Of Software Testing, chapter 2

Factors affecting intensity of
quality assurance activities in the
development process

QA activities in Development

- SQA Activities are linked to the completion of a project phase – Requirements, design, etc
- The SQA activities need to be **integrated** into the development plan that implements one or more software development models, such as the waterfall, prototyping, spiral, ...
- They need to be activities just like other more traditional activities – be entered in plan, scheduled, etc.

QA planning requirements

- SQA planner need to determine:
 - SQA activity list
 - Specify activity details:
 - Timing
 - Type
 - Who executes?
 - Internal: Development team, Department staff
 - External: Independent QA teams, Consultants
 - Resources for Corrections:
 - Defect removal
 - Introduction of changes

Factors affecting the required intensity of QA activities

- Project Factors
 - Magnitude of the project – how big is it?
 - Technical complexity and difficulty
 - Extent of reusable software components – a real factor
 - Severity of failure outcomes if project fails – essential!
- Team Factors
 - Professional qualifications of the team members
 - Team acquaintance with project and experience in the area
 - Availability of staff members who can professionally support the team, and
 - Percentage of new staff members in the team

Testing level

Quality Assurance vs. Testing Activities

Quality Assurance:

Multiple activities throughout the dev process

- Development standards
- Version control
- Change/Configuration management
- Release management
- **Testing** (as a subset of QA)
- Quality measurement
- Defect analysis
- Training

Testing:

Specific execution activities

- Unit testing
- Integration testing
- System testing
- Acceptance testing
- ...

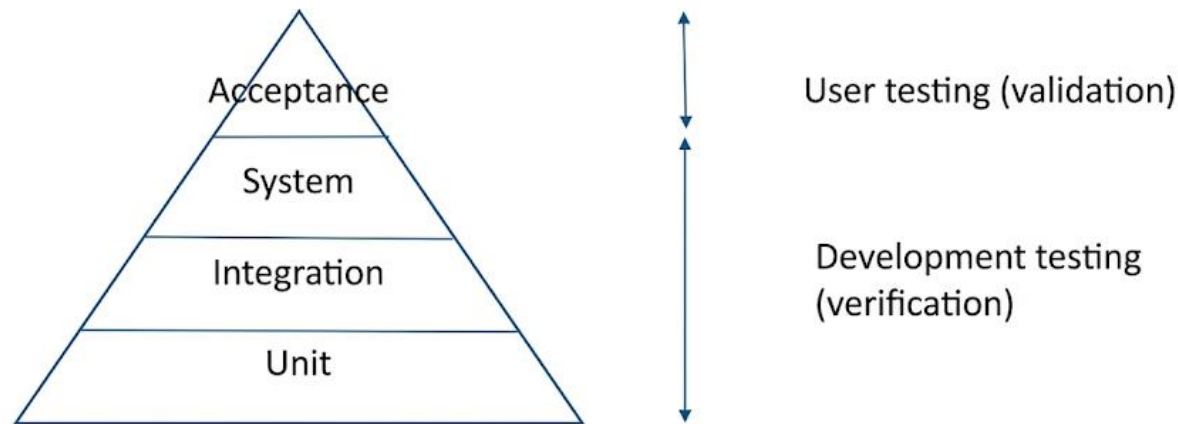
Testing Principles

1. Programmers should not test their own code
2. Testing should be performed by an independent role or team whenever possible.
3. The "Positive Pessimism" Mindset
4. Probability of more errors
5. Intellectually Challenging
6. Mandatory for every test case: Expected Output
7. Beyond the "Happy Path"
8. Verify what it **should not** do
9. Meticulous result checking
10. Design for Regression

Testing Level

Testing has 4 test levels:

- Unit testing
- Integration testing
- System testing
- Acceptance testing



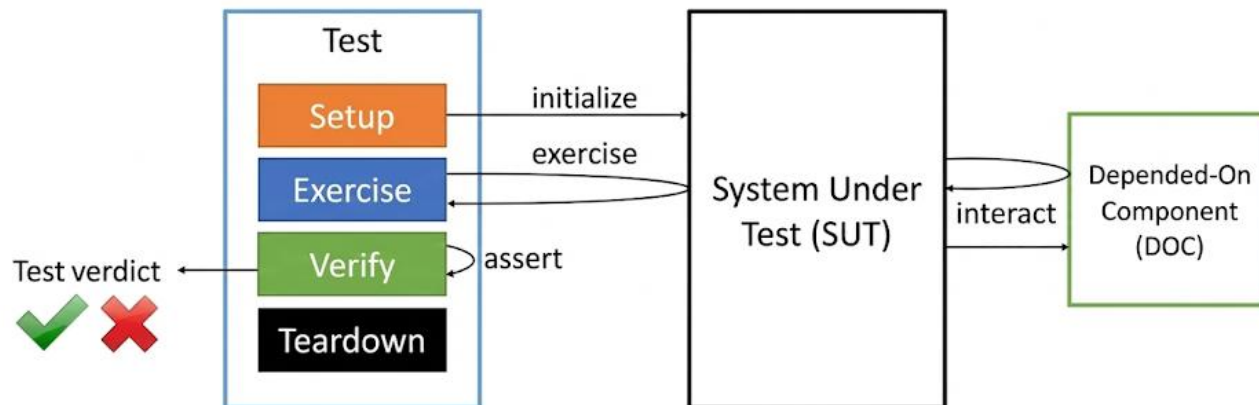
Testing levels

Testing Level

Unit testing: source code are tested to verify the design has been implemented

Unit test case has 4 phased:

- Set up
- Exercise
- Verify
- Teardown



Unit test generic structure

Testing Level

Integration testing: Test the interfaces, and the interaction between integrated components or modules

Strategies for performing integration testing:

- Top-down
- Bottom-up

Top-down integration:

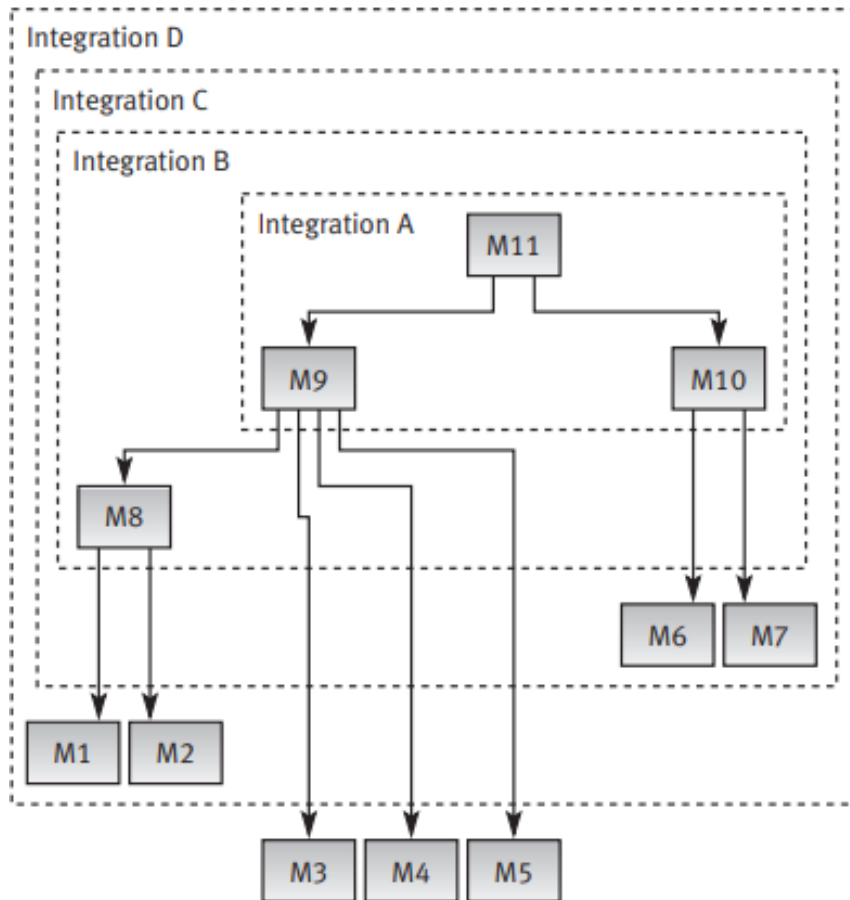
- Modules are integrated by moving downward through the control hierarchy, beginning with the main program.
- Modules subordinate to the main control module are incorporated into the structure in either a depth first or breadth-first manner.

Bottom-Up integration:

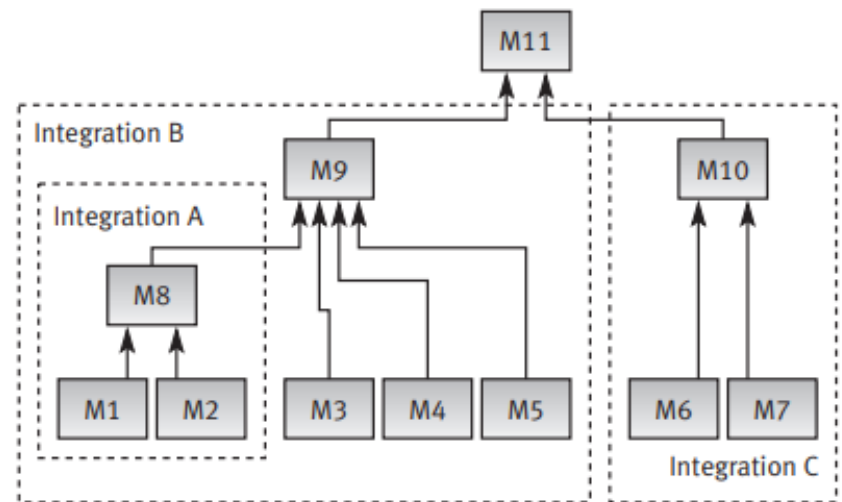
Begins construction and testing with components at the lowest levels in the program structure

Testing Level

- Top-down



- Bottom-up



Testing Level

System testing:

- Involves integrating components to create a version of the system and the testing the integrated system
- It verifies that the components are compatible, interacts correctly, and transfer the right data at the right time, topically across its user interfaces
- It overlaps with integration testing, but the difference here is that system testing should involve all the system components

Testing Level

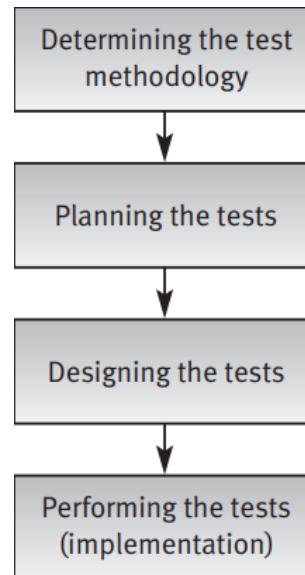
Acceptance testing:

- Test all system with real data or customer data
- The system development team should not be responsible for acceptance testing
- Checking to ensure that a system meets its requirements and is good enough for use by system customers
- Usually a black-box testing

Testing Process

The 4 Main Phases:

- Determining the test methodology
- Planning the tests
- Test design
- Test implementation



The testing process

Determining Test Methodology

Software Quality Standards:

- The required standard depends on the characteristics of the application and the magnitude of expected damage in case of failure.
- *Example:* Medical software requires the highest standard, whereas internal training software may only require a medium level

Testing Strategy Decisions:

- Should the strategy be "Big Bang" or "Incremental"?.
- Should testing be Bottom-up or Top-down?.
- Which parts require White Box testing or Automated testing models?

Planning the Tests

The Testing Hierarchy:

- Unit Tests
- Integration Tests
- System Tests

5 Core Planning Questions:

- What to test?
- Which sources to use for test cases?
- Who is to perform the tests?
- Where to perform the tests?
- When to terminate the tests?

Planning the Tests - Prioritization

- It is rarely feasible to test "everything," so priorities must be set
- Rating Method: Modules are rated based on a combined rating C :

$$C = kA + mB$$

- **Factor A (Severity):** Damage to life, finance, or essential functions
- **Factor B (Risk):** Probability of failure based on complexity and programmer experience

Example

Application	Damage severity level A	Risk severity level B	Combined rating method*		
			A + B	$7 \times A + 2 \times B$	$A \times B$
1. Input of test results	3	2	5 (4–5)	25 (5)	6 (4)
2. Interface for input and output of pupils' data to and from other teachers	4	4	8 (1)	36 (1)	16 (1)
3. Preparation of lists of low achievers	2	2	4 (6–7)	18 (7)	4 (5-6)
4. Printing letters to parents of low achievers	1	2	3 (8)	11 (8)	2 (8)
5. Preparation of reports for school principal	3	3	6 (3)	27 (4)	9 (3)
6. Display of a pupil's achievements profile	4	3	7 (2)	34 (2)	12 (2)
7. Printing of pupil's term report card	3	1	4 (6–7)	23 (6)	3 (7)
8. Printing of pupil's year-end report card	4	1	5 (4–5)	30 (3)	4 (5–6)

*The figures in brackets in the combined rating columns represent the application preferences.

Planning the Tests - Logistics

- **Test Case Sources:** Planners must decide between using samples of real-life test cases, synthetic test cases, or a combination of both
- **Who Performs Tests:**
 - Unit/Integration: Usually performed by the software development team
 - System Tests: Usually performed by an independent testing team (internal or external consultants)
- **Where to Test:**
 - Unit/Integration is done at the developer's site
 - System tests may be performed at the customer's site (target site) to simulate the actual environment

Planning the Tests - Termination Criteria

Deciding when to stop testing can follow one of five routes:

- **Completed Implementation:** All tests executed and clean
- **Mathematical Models:** Error detection rate drops to acceptable level
- **Error Seeding:** Sufficient % of artificially seeded errors found
- **Dual Teams:** Comparison of two independent team's results
- **Resource Limit:** Budget or time exhaustion

Design & Implementation

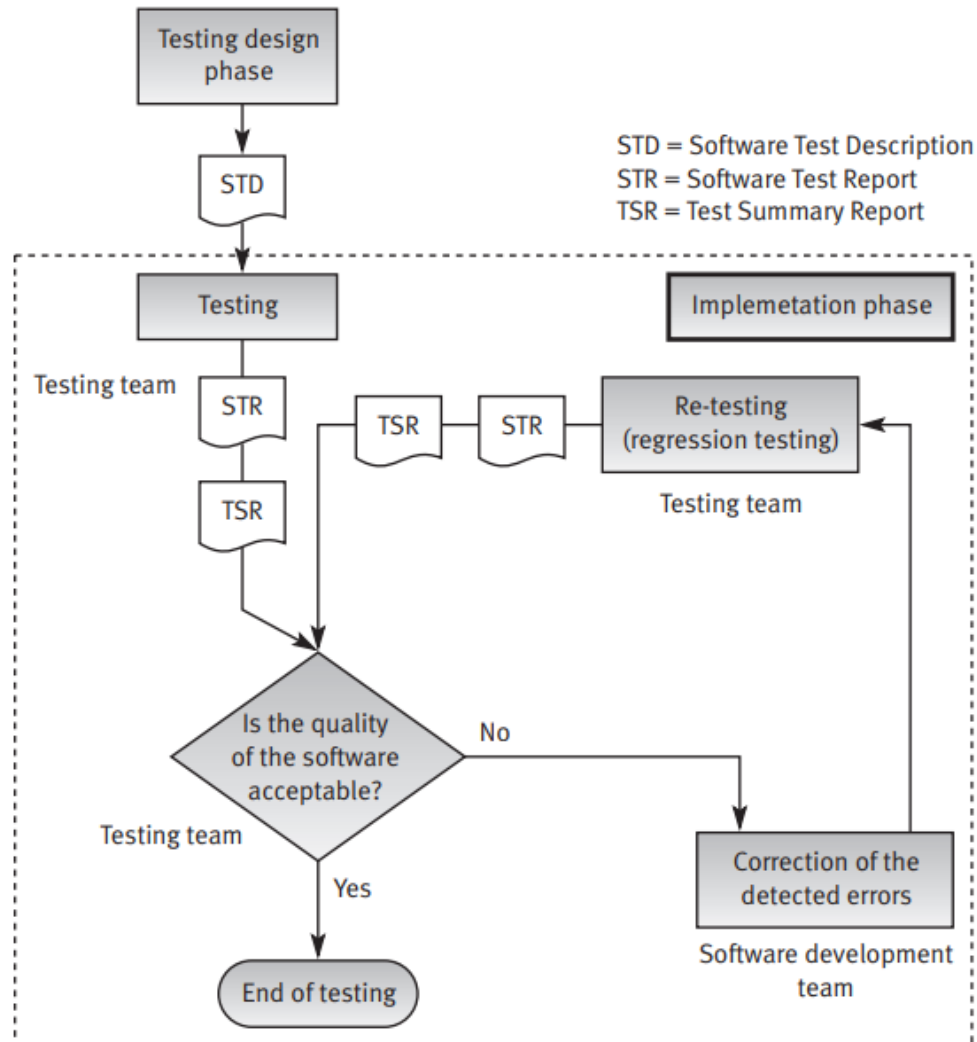
Test Design:

- Detailed design and procedures for each test
- Test Case Database/file

Implementation: Run tests → Fix errors → Regression Test

- Regression Test: Re-test to ensure fixes work and didn't break other parts

Testing Implementation



Implementation phase activities

Key Documentation

The process is supported by specific documentation at each stage:

- **STP (Software Test Plan):** The output of the planning stage.
- **STD (Software Test Description):** Documents the procedures and test cases.
- **STR (Software Test Report):** Documents the results of tests and re-tests.
- **TSR (Test Summary Report):** Summarizes the entire testing project.

A model for SQA defect removal effectiveness and cost

Objectives of the Model

- Purpose: To evaluate SQA plans based on two quantitative aspects:
 1. Total Effectiveness: The plan's ability to remove defects throughout the process
 2. Total Cost: The resources required to remove these defects
- Goal: To demonstrate the economic value and quality impact of investing in early SQA activities
- Context: The model typically assumes a linear software development process (Waterfall Model)

Key Data Inputs

To operate the model, three types of historical data are required (based on IBM, TRW, and Boehm statistics):

- 1. Defect Origin Distribution:** At which phase are defects introduced?
- 2. Defect Removal Effectiveness:** What percentage of defects does each QA activity find?
- 3. Cost of Defect Removal:** How much does it cost to fix a defect at each specific phase?

Defect Origin Distribution

Concept: Defects are introduced throughout the entire development lifecycle, not just during coding

No.	Software development phase	Average percentage of defects originating in phase
1	Requirements specification	15%
2	Design	35%
3	Coding (coding 30%, integration 10%)	40%
4	Documentation	10%

A characteristic distribution of software defect origins

Defect Removal Effectiveness

Concept: QA activities act as "filters." No single activity filters 100% of defects

No.	Quality assurance activity	Average defect filtering effectiveness rate
1	Requirements specification review	50%
2	Design inspection	60%
3	Design review	50%
4	Code inspection	65%
5	Unit test	50%
6	Unit test after code inspection	30%
7	Integration test	50%
8	System tests / acceptance tests	50%
9	Documentation review	50%

Average filtering (defect removal) effectiveness by quality assurance activities

Cost of Defect Removal

Concept: The cost to fix a bug rises exponentially as the project progresses

*Fixing a bug after release is **110x** more expensive than fixing it during requirements*

No.	Software development phase	Average relative defect cost (cost units)
1	Requirements specification	1
2	Design	2.5
3	Unit tests	6.5
4	Integration tests	16
5	System tests / acceptance tests / system documentation review	40
6	Operation by customer (after release)	110

Representative average relative defect-removal costs

How the Model Works

Mechanism:

- **Input Defects:** Sum of "New Defects" (from current phase) + "Passed Defects" (leaked from previous phase).
- **Filtering:**
 - *Removed Defects* = Input Defects $\times$ % Effectiveness.
 - *Passed Defects* = Input Defects - Removed Defects.
- **Cost Calculation:**
 - *Total Removal Cost (TRC)* = Removed Defects $\times$ Cost Unit (per phase)

Comparison: Standard vs. Comprehensive Plan

No.	Quality assurance activity	Defect removal effectiveness	Cost of removing a detected defect (cost units)
1	Requirement specification review	50%	1
2	Design review	50%	2.5
3	Unit test – code	50%	6.5
4	Integration test	50%	16
5	Documentation review	50%	16
6	System test	50%	40
7	Operation phase	100%	110

Standard quality assurance plan

No.	Quality assurance activity	Defect-removal effectiveness	Cost of removing a detected defect (cost units)
1	Requirement specification review	60%	1
2	Design inspection	70%	2.5
3	Design review	60%	2.5
4	Code inspection	70%	6.5
5	Unit test – code	40%	6.5
6	Integration test	60%	16
7	Documentation review	60%	16
8	System test	60%	40
9	Operation phase	100%	110

Comprehensive quality assurance plan

Comparison: Standard vs. Comprehensive Plan

No.	Quality assurance activity	Standard plan		Comprehensive plan	
		Percentage of removed defects	Cost of removing defects (cost units)	Percentage of removed defects	Cost of removing defects (cost units)
1	Requirements specification review	7.5%	7.5	9%	9
2	Design inspection	—	—	28.7%	71.8
3	Design review	21.3%	53.2	7.4%	18.5
4	Code inspection	—	—	24.4%	158.6
5	Unit test – code	25.6%	166.4	4.2%	27.3
6	Integration test	17.8%	284.8	9.8%	156.8
7	Documentation review	13.9%	222.4	9.9%	158.4
8	System test	7.0%	280	4%	160
	Total for internal quality assurance activities	93.1%	1014.3	97.4%	760.4
	Defects detected during operation	6.9%	759	2.6%	286
	Total	100.0%	1773.3	100.0%	1046.4

Comparison of the standard and comprehensive quality assurance plan

Summary

- Factors affecting intensity of quality assurance activities
- Testing level
- A model for SQA defect removal effectiveness and cost